

# Singleton Pattern

È un design pattern

Vogliamo fare in modo che di una classe possa esistere un'unica istanza

```
public class MiaClasse {
```

```
    private static MiaClasse m;
```

esiste a prescindere  
dell'esistenza di  
oggetti di tipo MiaClasse

```
    private MiaClasse() { }
```

non può essere chiamato  
da fuori MiaClasse

```
    public static MiaClasse getInstance() {
```

```
        if (m == null)
```

```
            m = new MiaClasse();
```

crea l'unica  
istanza possibile

```
        return m;
```

```
    }
```

```
}
```

In un'altra classe:

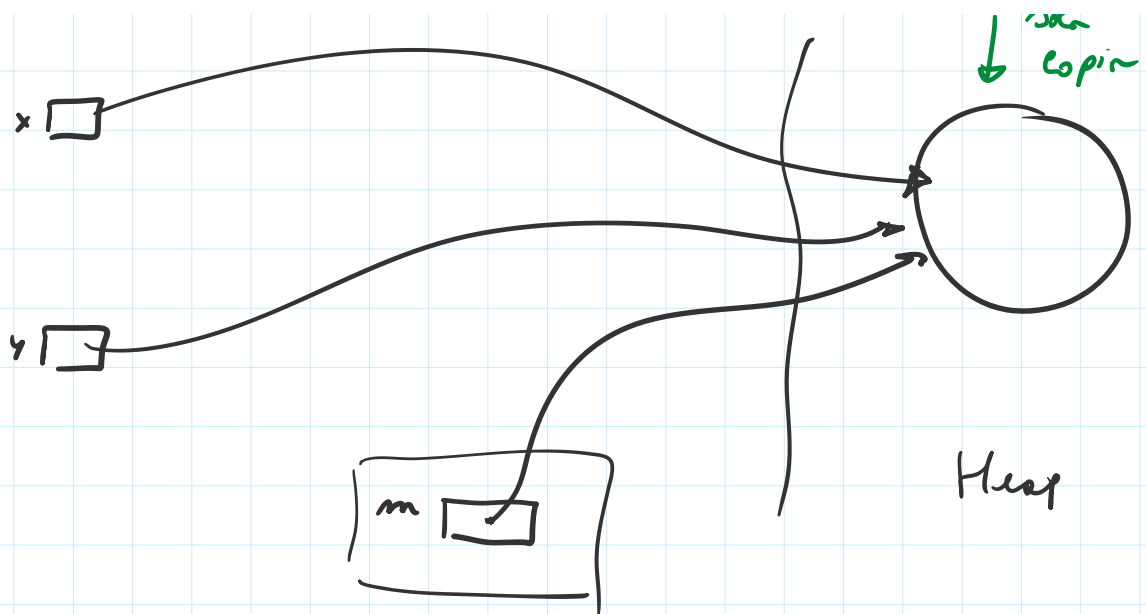
```
...
```

```
MiaClasse x = MiaClasse.getInstance();
```

```
...
```

```
MiaClasse y = MiaClasse.getInstance();
```

una  
↓ sola  
copia



MiaClasse z = new MiaClasse(); // errore

Il costruttore di default è pubblico

## Classi e file

Regola generale: ogni classe è definita in un proprio file e il file ha lo stesso nome della classe

Cosa succede quando compiliamo con javac una classe che ne usa altre:

```
public class MiaClasse {
    ;
}
```

→ MiaClasse.java

```
public class Principale {
    :
    MiaClasse x = new MiaClasse();
    :
}
```

} → Principale.java

compiliamo Principale

\$ javac Principale.java

- 1) il compilatore si accorge che Principale usa MiaClasse
- 2) il compilatore cerca MiaClasse.class
  - a) se MiaClasse.class non c'è javac compila MiaClasse.java
  - b) se MiaClasse.class è presente javac controlla la data di file MiaClasse.class e MiaClasse.java, nel caso in cui il file sorgente sia più recente lo ricompila.

Regola: all'interno di un unico file è possibile memorizzare la definizione di più classi; al più una classe può essere indicata come "public"

Regola: se c'è una classe "public" allora il file sorgente deve avere lo stesso nome della classe

Esempio

```
class Uno {  
    :  
}  
  
class Due {  
    :  
}  
  
public class Tre {  
    :  
}
```

→ Tre.java

```
$ javac Tre.java  
produce 3 file .class:  
    Uno.class  
    Due.class  
    Tre.class
```

Quando seguire questo approccio?

solo quando le classi Uno e Due  
(quelle non public) vengono usate  
solo ed esclusivamente da Tre

Perché?

supponiamo di avere una classe

Quattro che usa Uno

cercò di compilare Quattro

```
$ javac Quattro.java
```

javac cerca Uno.class

se non c'è, cerca Uno.java

se non c'è, cerca Uno.java  
che però non esiste (Uno è definito in Tre.java)  
la compilazione si ferma con una segnalazione  
di errore.

Se invece Tre.java è stato compilato in  
precedenza allora Uno.class viene trovato.

### Class, file e filesystem (directory)

Se definisci una classe che appartiene a un  
package il file sorgente deve metterlo  
in una cartella che ha lo stesso nome  
del package.

Supponiamo di definire X.java  
e X appartiene al package abc.def

abc/

abc/  
└─ def/  
    └─ X.java

per compilare mi devo "posizionare" nella cartella che contiene la cartella "abc"

```
$ javac abc/def/X.java
```

produce X.class nella cartella def

↑ abc/  
   └─ def/  
       └─ X.java  
          └─ X.class

Se X contiene un main, per mandarlo in esecuzione mi devo "posizionare" nella cartella che contiene la cartella "abc" e dare il comando

```
$ java abc.def.X
```

la JVM cerca il file X.class nella cartella "def" contenuta nella cartella "abc"

Regola: posizionarsi, sia per compilazione che per esecuzione, nella cartella che contiene la radice del package

`abc/`  
 $\hookrightarrow$  `def/`  
 $\hookrightarrow$  `X.java`  
 $\hookrightarrow$  `X.class`

Qui  
↓

`c1/`  
 $\hookrightarrow$  `c2/`  
 $\hookrightarrow$  `c3/`  
 $\hookrightarrow$  `Y.java`  
 $\hookrightarrow$  `Y.class`

Se `Y` che appartiene al package `c1.c2.c3` usa `X` (che appartiene a `abc.def`)

con il comando

`$ javac c1/c2/c3/Y.java`

viene prodotto.

`Y.class`  
`X.class`

## classpath

`classpath`: un insieme di cartelle\* a partire dalle quali il compilatore e la JVM cercano le classi che servono loro

\* possono essere non solo cartelle ma anche file `.jar` (archivi)



Se il `classpath` non viene specificato allora

Se il classpath non viene specificato allora è uguale a "."

Possiamo specificarlo con le opzioni

-cp oppure -classpath

i vari punti di partenza possono essere separati con : in Unix e ; in Win

java -classpath /home/mario:/home/luigi abc.Z

la JVM cerca

Z.class in

e in

/home/mario/abc

/home/luigi/abc

lo stesso vale per il compilatore

java -classpath .:/var/lib abc.Z

Le classi di sistema vengono trovate automaticamente

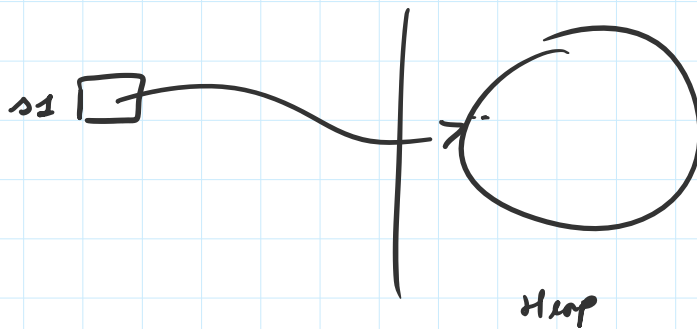
## Gestione della memoria

Tutti gli oggetti vengono allocati nello heap

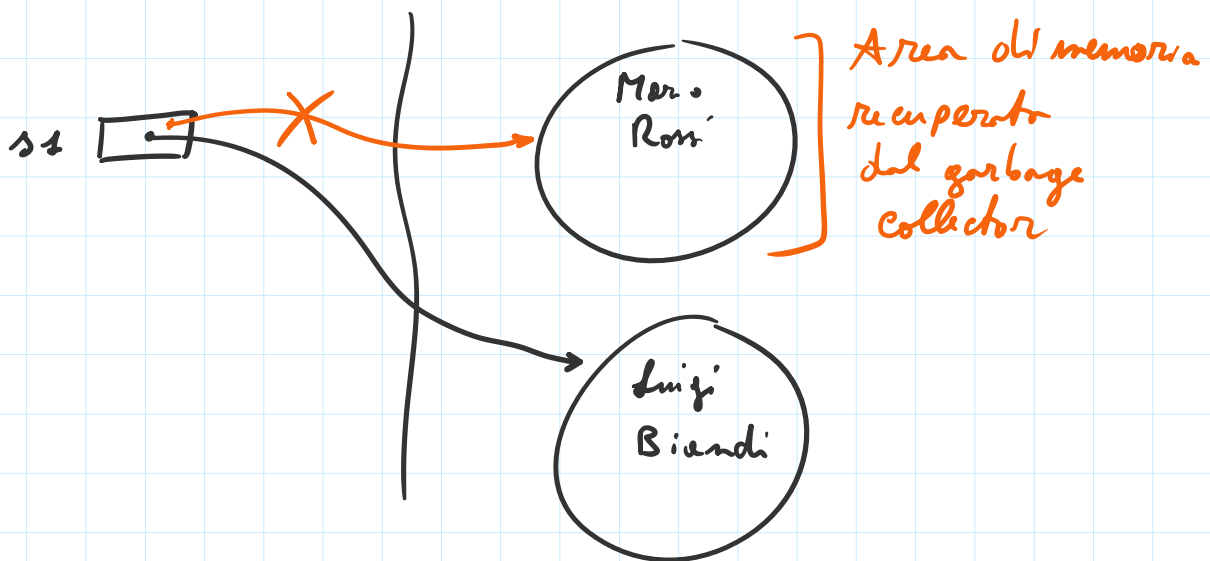
La deallocazione degli oggetti non è responsabilità del programmatore

La JVM dispone di un sistema per il recupero della memoria non più utilizzata:  
il garbage collector

```
Studente s1 = new Studente("Mario", "Roni");
```



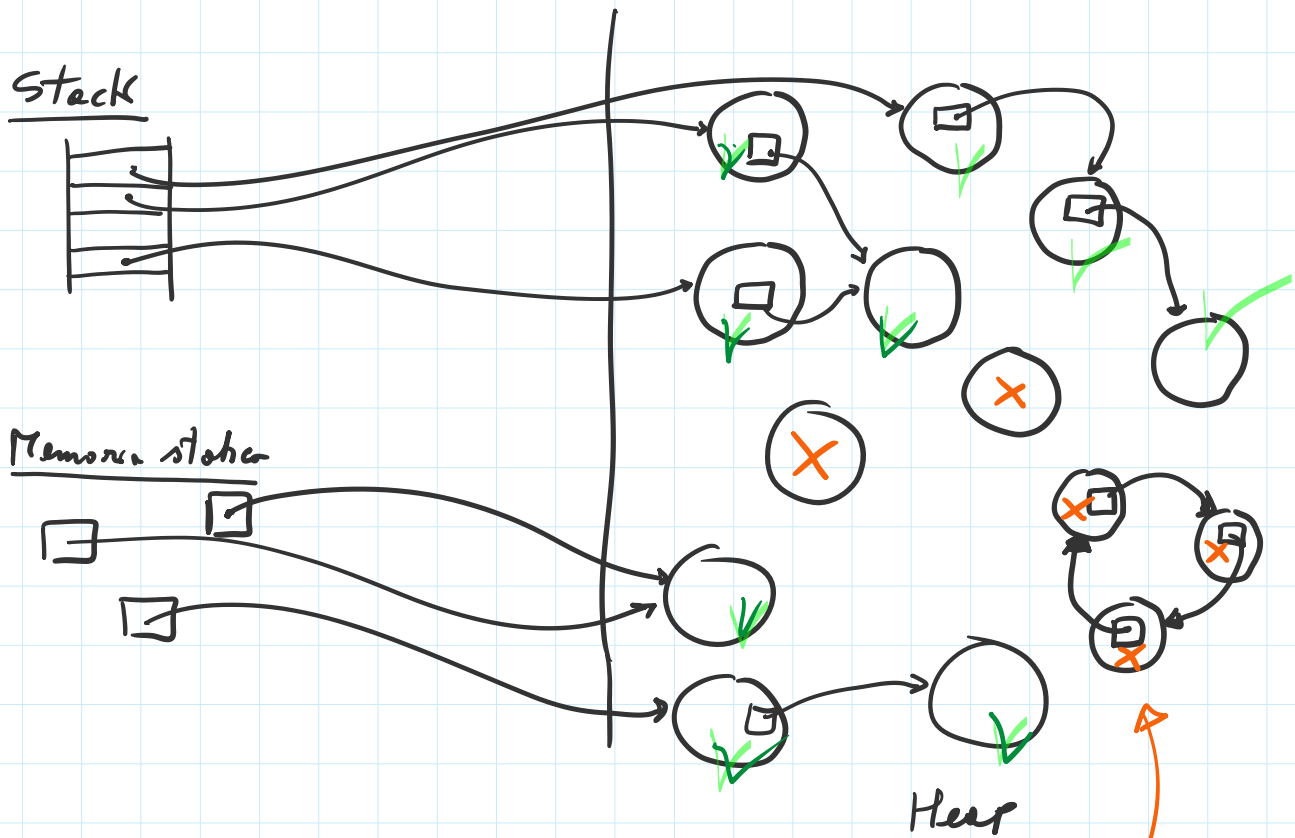
```
s1 = new Studente("Luigi", "Biendi");
```



Il garbage collector recupera la memoria quando vuole lui

Posso mandare in esecuzione il garbage collector con il metodo `gc()` della classe `System`

```
System.gc();
```

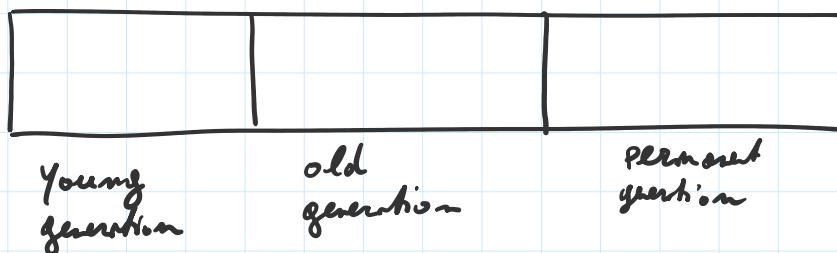


il g.c.  
è in grado di  
captare che non  
sono raggiungibili

Gli oggetti possono essere divisi in due categorie:

- quelli che vivono poco
- quelli che vivono molto

G.C. generazionali



Young generation

S: Survivor

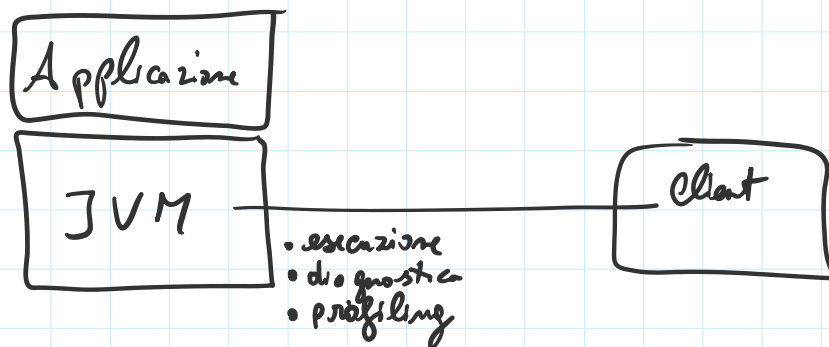
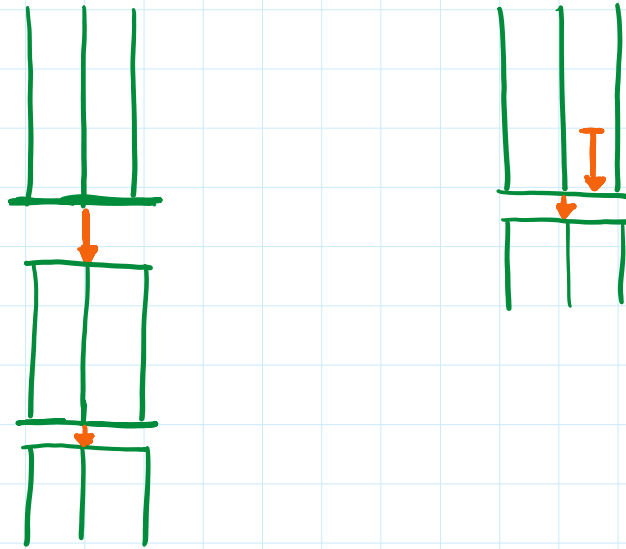


Quando il contatore supera un certo valore l'oggetto viene spostato nella old generation

Minor garbage collection (solo sulla young generation)

Major garbage collection (quando la old generation è piena)

Molte attività del G.C. sono del tipo "stop-the-world"



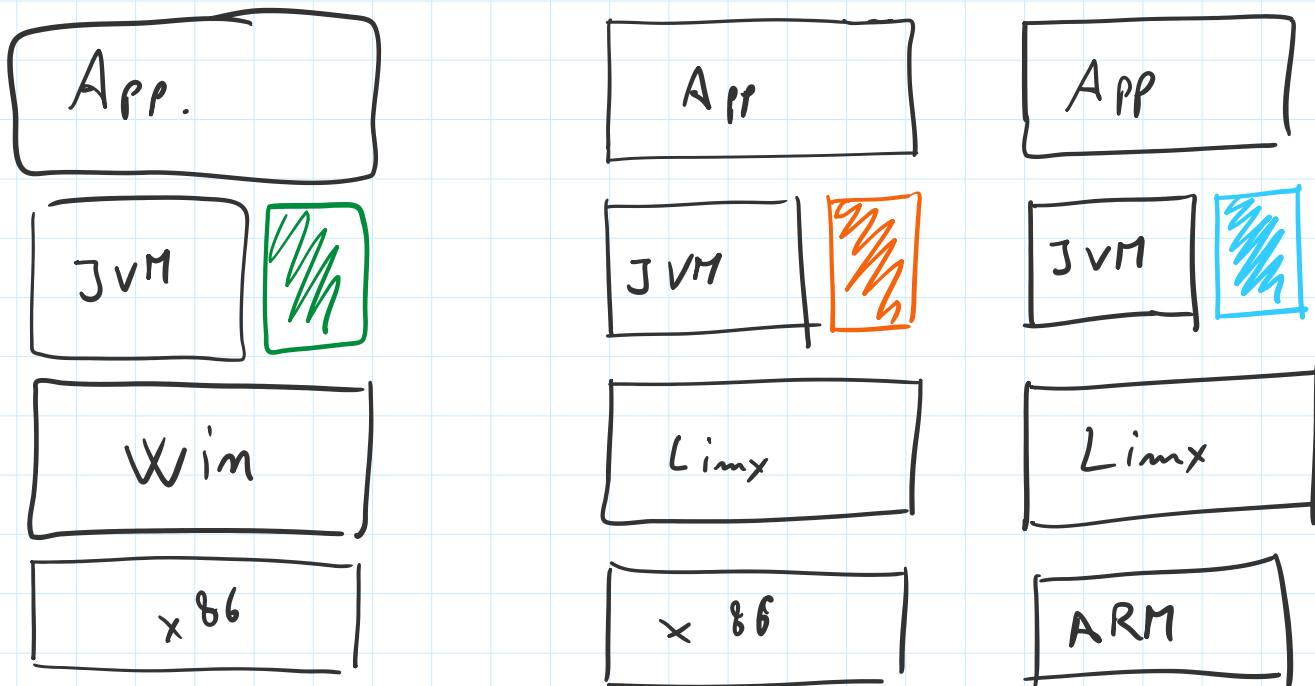
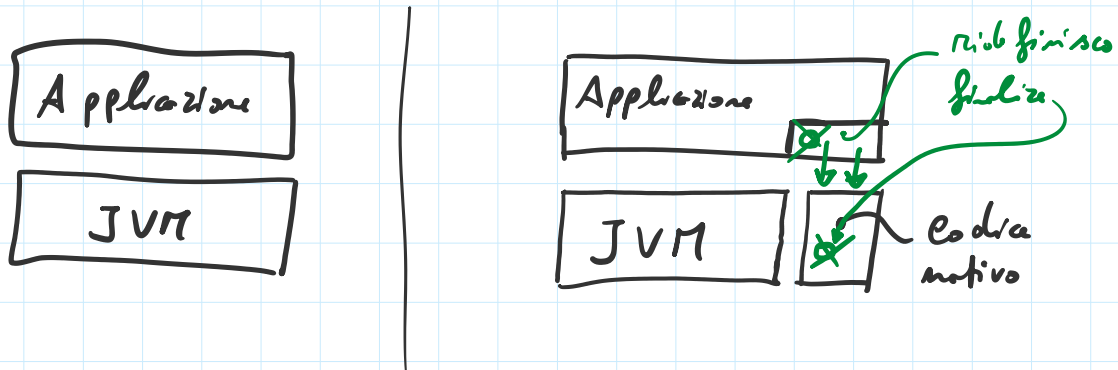
java -classpath /var/lib/oracle.jar: ~

Per ricevere "notifiche" dal g.c. quando un oggetto sta per essere raccolto

ridefinire il metodo

protected void finalize()

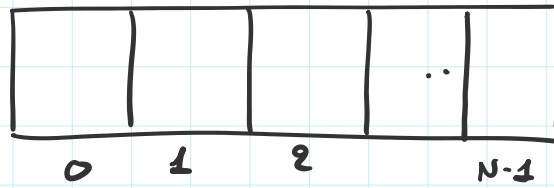
utile soprattutto se il codice Java è  
collegato a codice non Java (codice nativo)



## Array

Contenitore di valori dello stesso tipo  
elementi identificati da un indice

indici partono da zero



↖ con N dimensioni dell'array

Dimensione definita a tempo di esecuzione

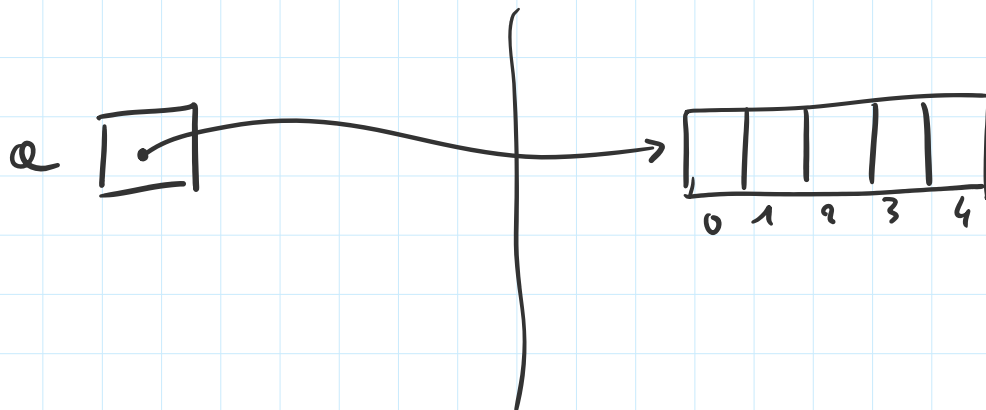
`int[] a;`

Ho creato un riferimento ad array di int  
per ora non c'è nessun array

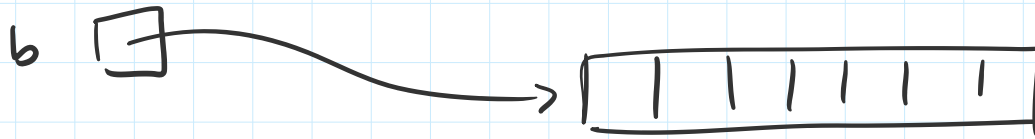
`a = new int[5];`

adesso ho creato un array di 5 int  
memorizzato nello heap

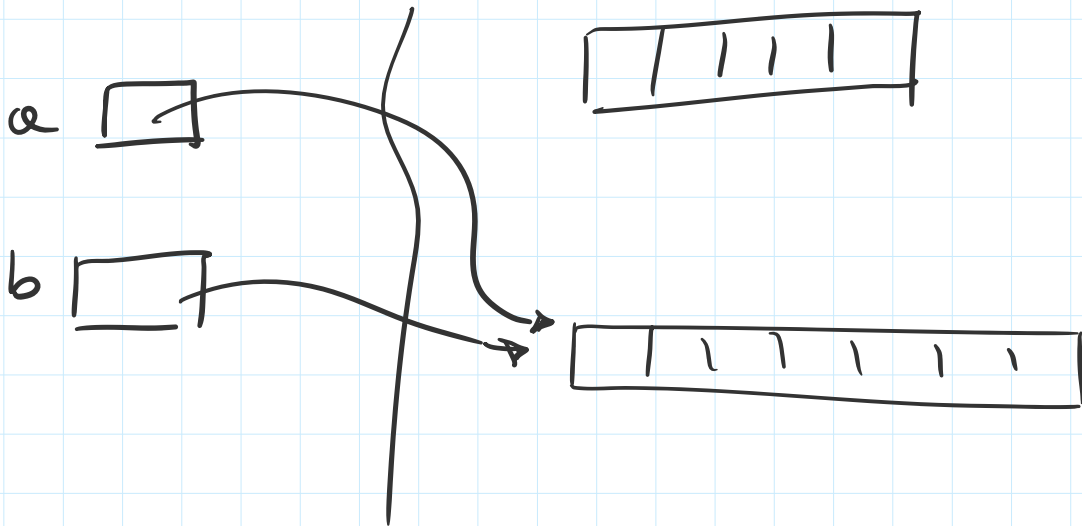
lo posso raggiungere con il riferimento a



```
int[] b = new int[10];
```



```
a = b; // OK
```

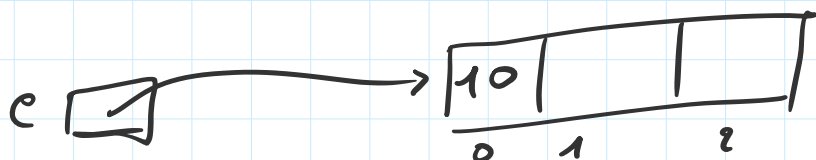


per accedere, selezione con indice

```
int[] c = new int[3];
```

```
c[0] = 10;
```

```
int k = c[0];
```



In generale

```
Tipo[] nome = new Tipo[dimensione];
```

variabile membro `length` che me indica  
la lunghezza

```
int[] a1 = new int[5];
```

```
int l = a1.length; // length: lunghezza  
// l vale 5
```

```
float[] a2 = new float[23];
```

```
l = a2.length; // l vale 23
```